AgenticTrip — Phase 1 Architecture & Execution Plan

Project Vision

AgenticTrip is an AI-powered travel planning and execution platform that uses autonomous AI agents, MCP integrations, real-time APIs, memory, and conversational workflows to:

- Understand user travel goals naturally
- Plan complete trips end-to-end
- Book and manage travel services
- Provide real-time recommendations
- Adapt dynamically during travel
- Support multilingual interaction
- Operate as a multi-agent ecosystem

High-Level Architecture

```
Frontend (React/Vite)
        ↓
FastAPI Backend Gateway
        ↓
Agent Orchestrator Layer
        ↓
-------------------------------------------------
| Planner Agent | Budget Agent | Booking Agent |
-------------------------------------------------
| Weather Agent | Maps Agent   | Support Agent |
-------------------------------------------------
        ↓
Memory + RAG + MCP Tool Layer
        ↓
External APIs / Services
```

Core Technology Stack

Frontend

- React + Vite
- TailwindCSS
- ShadCN UI
- Framer Motion

- Zustand or Redux Toolkit

Backend

- FastAPI
- Python 3.11+
- Uvicorn
- Pydantic

AI / Agent Framework

Recommended:

Option 1 (Recommended)

- LangGraph
- LangChain
- Azure OpenAI

Option 2

- CrewAI

Option 3

- Semantic Kernel

Recommended AI Models

Primary LLM

- Azure OpenAI GPT-4.1

Embedding Model

- text-embedding-3-large

Speech Models (Future)

- Whisper
- Azure Speech

Phase-wise Development Plan

PHASE 1 — Foundation Setup

Goal

Create production-ready base architecture.

Deliverables

1. Frontend Setup

```
npm create vite@latest agentictrip
```

Install:

```
npm install tailwindcss react-router-dom axios
npm install framer-motion lucide-react
```

---

2. Backend Setup

```
mkdir backend
cd backend
python -m venv venv
```

Install:

```
pip install fastapi uvicorn python-dotenv
pip install openai langchain langgraph
pip install chromadb faiss-cpu
pip install tiktoken pydantic
```

---

3. Environment Configuration

Create `.env`

```
AZURE_OPENAI_API_KEY=
AZURE_OPENAI_ENDPOINT=
```

```
AZURE_OPENAI_DEPLOYMENT=
AZURE_OPENAI_API_VERSION=
```

---

4. Basic FastAPI Structure

```
backend/
│
├── app/
│   ├── agents/
│   ├── tools/
│   ├── memory/
│   ├── rag/
│   ├── api/
│   ├── services/
│   └── main.py
│
├── requirements.txt
└── .env
```

PHASE 2 — Multi-Agent System

Initial Agents

1. Planner Agent

Responsibilities:

- Understand travel intent
- Build itinerary
- Coordinate other agents

2. Budget Agent

Responsibilities:

- Estimate trip cost
- Suggest optimizations
- Currency conversion

3. Destination Agent

Responsibilities:

- Suggest attractions
- Recommend local experiences

- Generate travel insights

4. Booking Agent

Responsibilities:

- Flights
- Hotels
- Transportation

5. Weather Agent

Responsibilities:

- Forecast analysis
- Weather-based recommendations

6. Support Agent

Responsibilities:

- FAQs
- Emergency support
- Trip modifications

PHASE 3 — MCP Integration Layer

MCP Concept

MCP (Model Context Protocol) allows AI agents to use external tools/services in a standardized way.

MCP Integrations Needed

Capability	MCP / API
Maps	Google Maps API
Hotels	Booking.com API
Flights	Amadeus API
Weather	OpenWeatherMap API
Currency	ExchangeRate API
Search	Tavily / SerpAPI
Calendar	Google Calendar API

Capability	MCP / API
Email	Gmail API
Payments	Stripe

PHASE 4 — RAG + Memory

Goal

Allow agents to remember users and retrieve contextual travel data.

Long-Term Memory

Store:

- User preferences
- Past trips
- Budget patterns
- Favorite destinations
- Language preference

RAG Sources

Possible datasets:

- City guides
- Tourism PDFs
- Hotel reviews
- Airline policies
- Visa rules
- User uploaded documents

Vector DB Options

Recommended:

- ChromaDB
- FAISS

Future:

- Pinecone
- Weaviate

PHASE 5 — Real-Time Agent Workflows

Example Workflow

User:

> Plan a 5-day trip to Bali under ₹80,000

Agent Flow

```
Planner Agent
     ↓
Budget Agent
     ↓
Flight Search Tool
     ↓
Hotel Search Tool
     ↓
Weather Agent
     ↓
Final Travel Plan
```

PHASE 6 — UI/UX Features

Features

- Chat-based interface
- Dynamic itinerary cards
- Map visualization
- Expense tracker
- Real-time alerts
- Voice interaction
- Multi-language support
- AI travel assistant sidebar

Suggested Database

PostgreSQL

Tables:

- users

- trips
- itineraries
- expenses
- memories
- bookings
- conversations

Authentication

Recommended:

- Clerk OR
- Firebase Auth OR
- Auth0

Deployment Strategy

Frontend

- Azure Static Web Apps OR
- Vercel

Backend

- Azure App Service
- Azure Container Apps
- Dockerized deployment

Database

- Azure PostgreSQL

Vector DB

- Chroma local initially
- Pinecone later

Recommended Repository Structure

```
AgenticTrip/
│
├── frontend/
├── backend/
├── docs/
├── architecture/
├── prompts/
├── datasets/
└── deployment/
```

Immediate Action Items

STEP 1

Create GitHub repository:

```
AgenticTrip
```

STEP 2

Setup frontend + backend base project.

STEP 3

Create first Planner Agent.

STEP 4

Integrate Azure OpenAI.

STEP 5

Build simple conversational trip planner.

First Milestone Goal

By the end of Milestone 1, AgenticTrip should:

✅ Accept user trip requests ✅ Understand travel goals ✅ Generate travel itineraries ✅ Use at least 2 tools/APIs ✅ Maintain conversation memory ✅ Support multi-agent orchestration

---

Suggested Development Sequence

1. Base Architecture
2. Planner Agent
3. Tool Calling
4. Memory
5. RAG
6. Multi-Agent Coordination
7. Real-Time APIs
8. Deployment
9. Voice Support
10. Autonomous Workflows

Long-Term Vision

Future enhancements:

- Voice-enabled AI travel companion
- Autonomous booking approvals
- Real-time flight disruption handling
- AI-generated travel videos
- Group trip collaboration
- Expense splitting
- AR/VR travel previews
- AI concierge mode
- WhatsApp/Telegram integration

FastAPI Foundation Learning Notes

What We Built

We created the first working backend service for AgenticTrip using FastAPI and Uvicorn.

Current architecture:

```
Browser
   ↓
FastAPI Backend
   ↓
Python Logic
```

Future architecture:

```
React Frontend
      ↓
FastAPI Backend
      ↓
AI Agents
      ↓
Azure OpenAI + APIs + Databases
```

Understanding main.py

```python
from fastapi import FastAPI

app = FastAPI(
    title="AgenticTrip API",
    version="1.0.0"
)

@app.get("/")
def home():
    return {
        "message": "Welcome to AgenticTrip"
    }
```

---

1. Importing FastAPI

```python
from fastapi import FastAPI
```

This imports the FastAPI framework into the project.

Purpose:

- Use FastAPI features
- Create APIs
- Handle requests and responses

- Build backend services

---

2. Creating FastAPI Application

```
app = FastAPI(
    title="AgenticTrip API",
    version="1.0.0"
)
```

This creates the backend application instance.

Metadata:

- title → API name
- version → API version

These appear automatically inside Swagger documentation.

---

3. Creating API Endpoint

```
@app.get("/")
```

This defines an API route.

Meaning:

When someone sends a GET request to `/`, execute the function below.

Examples of future endpoints:

Endpoint	Purpose
`/`	Home
`/chat`	AI chat
`/trips`	Trip management
`/hotels`	Hotel search
`/weather`	Weather info

---

4. Route Function

```
def home():
```

This function runs when the endpoint is accessed.

---

5. Returning JSON Response

```
return {
    "message": "Welcome to AgenticTrip"
}
```

FastAPI automatically converts Python dictionaries into JSON responses.

Browser output:

```
{
  "message": "Welcome to AgenticTrip"
}
```

Understanding Uvicorn Command

Command used:

```
uvicorn app.main:app --reload
```

Breakdown

uvicorn

Runs the backend server.

---

app.main

Meaning:

- Open `app/` folder
- Open `main.py`

:app

Use the FastAPI object named `app`.

--reload

Automatically reloads the server whenever code changes.

Useful during development.

Swagger Documentation

URL:

```
http://127.0.0.1:8000/docs
```

FastAPI automatically generates:

- API documentation
- Endpoint testing interface
- OpenAPI schema
- Request/response visualization

Benefits:

- Easier frontend integration
- Better developer experience
- Enterprise-grade API management
- Faster testing

What FastAPI Automatically Provided

Without writing additional logic, FastAPI already gave:

✅ Running backend server ✅ API routing ✅ JSON handling ✅ Swagger documentation ✅ OpenAPI schema generation ✅ Validation support ✅ Scalable backend architecture

---

Current AgenticTrip Backend Flow

Current:

```
User → FastAPI → JSON Response
```

Future:

```
User
 ↓
React Frontend
 ↓
FastAPI Backend
 ↓
Planner Agent
 ↓
Azure OpenAI
 ↓
Travel APIs
 ↓
AI Response
```

Importance Of This Step

This was the first foundational backend service for AgenticTrip.

This structure will later support:

- AI agents
- Multi-agent orchestration
- Tool calling
- Azure OpenAI integration
- Memory systems
- RAG pipelines
- Real-time APIs
- Production deployment on Azure App Service

Frontend ↔ Backend Integration Notes

Objective

Connect React frontend with FastAPI backend using Axios.

Goal:

```
Frontend → Backend → Response → UI Update
```

Architecture After Integration

```
React Frontend
       ↓
Axios Request
       ↓
FastAPI Backend
       ↓
JSON Response
```

Future architecture:

```
Frontend
   ↓
FastAPI
   ↓
Planner Agent
   ↓
LangGraph
   ↓
Azure OpenAI
   ↓
Travel APIs
   ↓
Memory + RAG
```

Axios Installation

Installed Axios inside frontend.

Command:

```
npm install axios
```

Purpose:

- Send API requests
- Communicate with backend
- Receive JSON responses

API Service Layer

Created:

```
src/services/api.js
```

Code:

```
import axios from "axios"

const api = axios.create({
  baseURL: "http://127.0.0.1:8000"
})

export default api
```

Purpose:

- Centralized API configuration
- Reusable backend communication layer
- Cleaner frontend architecture

Backend Chat Endpoint

Updated:

```
backend/app/main.py
```

Added:

```
from fastapi import FastAPI
from pydantic import BaseModel
from fastapi.middleware.cors import CORSMiddleware
```

CORS Middleware

Added:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Purpose:

- Allow frontend and backend communication
- Resolve browser security restrictions
- Enable cross-origin requests

Understanding CORS

Frontend:

```
http://localhost:5173
```

Backend:

```
http://127.0.0.1:8000
```

Since ports differ, browser treats them as different origins.

Without CORS:

```
OPTIONS /chat 405 Method Not Allowed
```

appears.

CORS middleware resolved this issue.

Pydantic Request Model

```
class ChatRequest(BaseModel):
    message: str
```

Purpose:

- Validate frontend request data
- Ensure proper structure
- Prevent invalid requests

Chat Endpoint

```
@app.post("/chat")
def chat(request: ChatRequest):

    user_message = request.message

    return {
        "response": f"AgenticTrip AI received: {user_message}"
    }
```

Purpose:

- Receive frontend messages
- Process request data
- Return backend response

Frontend Home.jsx Integration

Implemented:

- React state management
- Axios API calls
- Dynamic UI updates
- Response rendering

Key concepts used:

React State

```
const [message, setMessage] = useState("")
const [response, setResponse] = useState("")
```

Purpose:

- Store user input
- Store backend response
- Automatically re-render UI

API Request Function

```
const handleSubmit = async () => {

  const res = await api.post("/chat", {
      message: message
  })

  setResponse(res.data.response)
}
```

Purpose:

- Send request to backend
- Receive response
- Update UI dynamically

Full Request Flow

When user clicks:

```
Plan Trip
```

This flow occurs:

```
React UI
   ↓
Axios POST Request
   ↓
FastAPI /chat endpoint
   ↓
Pydantic validation
   ↓
Backend processing
   ↓
JSON response
   ↓
React state update
   ↓
UI re-render
```

Result Achieved

Successfully implemented:

✅ React frontend ✅ FastAPI backend ✅ Axios integration ✅ API communication ✅ POST request handling ✅ Pydantic validation ✅ CORS handling ✅ Dynamic UI rendering ✅ Full-stack workflow foundation

---

Current AgenticTrip System

```
Frontend (React)
       ↓
Axios API Layer
       ↓
FastAPI Backend
       ↓
Business Logic
```

Importance Of This Milestone

This is the foundational architecture for:

- AI chat systems
- AI copilots
- Agentic workflows
- SaaS platforms
- Enterprise AI applications

This same request pipeline will later support:

- Azure OpenAI
- LangGraph agents
- Travel APIs
- Memory systems
- RAG pipelines
- Autonomous workflows

UI Components Built So Far

Frontend currently contains:

```
components/
└── Navbar.jsx
```

Pages:

```
pages/
└── Home.jsx
```

Layouts:

```
layouts/
└── MainLayout.jsx
```

Current UI Features

Implemented:

✅ Premium dark theme ✅ Navbar ✅ Hero section ✅ AI prompt box ✅ Responsive centered layout ✅ Dynamic response card

---

Next Recommended Tasks

Upcoming frontend improvements:

- gradients
- glow effects
- loading animations
- typing indicator
- glassmorphism
- responsive optimization
- chat history
- better cards
- Enter-key submission
- error handling

Upcoming backend improvements:

- Azure OpenAI integration
- Planner Agent
- LangGraph orchestration
- memory system
- RAG architecture

Session Stopping Point

AgenticTrip is now officially a working full-stack application.

Current status:

✅ Frontend working ✅ Backend working ✅ API communication working ✅ Full-stack architecture established

Next session will continue from UI enhancement and AI integration phase.

Recommended Next Task

Build the foundational FastAPI backend with:

- Azure OpenAI connection
- Planner Agent
- Tool architecture
- Simple travel chat endpoint
- Memory layer

This should be the next implementation step.